Training Teams Flow (Working Draft)

> **Purpose of this doc**: capture the end-to-end flow of how Magi/Gemini evaluation data is produced by pods (human teams + auto-rater), so our pipeline can ingest, standardize, and later generate (a) training datasets and (b) analytics datasets.
>
> **You can edit anything** in this doc — especially the ✅/⚠️ notes and the sections marked **[EDIT HERE]**.

---

1) Pod structure (who produces the data)

1.1 Vertical (Domain) Pods

Domain-specific human pods that evaluate tasks in a specific area.

- Examples: `Magi_Code_Python`, `Magi_Math`, `Magi_English`, `Magi_Shopping`, `Magi_Maps`

1.2 Horizontal (Cross-cutting) Pods

Quality/safety pods that evaluate cross-domain dimensions.

- Examples: `Magi_Safety`, `Magi_Factuality`, `Magi_Reasoning`

✅ **Clarification (from user):** each pod is a **distinct human training/evaluation team**.

---

2) Training cycle: batches are created per pod

When the main Magi/Gemini model needs training, the system generates **multiple batches**, where **each batch is tied to a specific pod**.

- A pod can receive multiple batches per training cycle.
- Each batch contains many evaluation units that originate from model runs.

✅ **Clarification (from user):** “Every time the main model needs training, a batch of data is created for that specific pod; multiple batches are created overall.”

---

3) Batch composition: Auto-rater + human sampled sets

Each batch contains **100% of candidate examples** (e.g., 100,000 items). This batch is split into:

- **Auto-rater path (scale):** \~90% (e.g., 90,000)

- sent to automated LLM-based evaluation (Auto-rater/Judge)

- **Human path (gold):** \~10% (e.g., 10,000)

- sampled and divided into **multiple "sets"** (example: 5 sets × 2,000 rows)

✅ **Clarification (from user):** "10k is sampled as multiple sets like 5 sets with 2k each."

Edited: if sampling ratios vary by pod/task:** Yes the sampling ratio can be varied, varied in overall percentages, the counts provided are just for your understand so the counts can also be varied, the number of sets can be varied (not fixed numbers), each set count can be varied not fixed 2000 rows,

- Auto-rater %:
- Human %:
- Number of sets per batch:
- Rows per set:

---

4) What a prompt produces: execution steps (variable length)

For each **user prompt**, the model may execute a variable number of steps to reach the final answer.

- Some prompts have **3 steps**, others **5**, others **10+**.
- Steps depend on:
- the user prompt
- pod/tooling
- search/tool usage

Each prompt ends with a final step where the model emits the final answer.

✅ **Clarification (from user):** "All we care is evaluating the model at each step."

---

5) Core entities and how they relate (IMPORTANT)

5.1 Batch

A batch is the unit created per pod per training cycle.

- `batch_id`, `batch_name`, `pod_name`, `task_type`, `model_version`, `export_ts`, etc.

5.2 Set

A set is a human evaluation workload container inside a batch.

- Example: 5 sets per batch.

- Each set contains many **rows**, where a row corresponds to a **(prompt × step)** evaluation.

✅ **Clarification (from user):** "A particular SET which has 2k rows means total unique search query × total number of steps for that query."

5.3 Row (the atomic evaluation unit)

This is the key correction.

A **row** represents one evaluation point:

- one prompt
- one step number in its execution trace
- step content (tool call/search query/tool output/partial reasoning/etc.)
- possibly final answer only at the last step

So:

- `row_id` (or computed)
- `prompt_id` / `query_id`
- `step_index`
- `step_type` (search/tool/final/etc.)
- `step_payload`

✅ **Clarification (from user):** "Each row will be assigned to 2 curators for sure and 1 reviewer for sure."

---

6) Rating workflow per row (curator + reviewer)

For **every row** (prompt-step), we have:

6.1 Curator ratings (always 2)

- Curator 1 produces: `curator_1_rating`
- Curator 2 produces: `curator_2_rating`

Ratings include rubric dimensions such as:

- Primary Intent
- Information Gain
- Reasoning
- Understanding
- Implementation
- Arguments
- Trajectory Robustness
- (others depending on pod)

Violations can also be tagged.

6.2 Reviewer edits (always 1 reviewer)

Reviewer can **edit/override** curator scores (not create a brand-new separate rubric).

So we store:

- `reviewer_curator_1_rating` (edited/confirmed version of curator 1)
- `reviewer_curator_2_rating` (edited/confirmed version of curator 2)

Same applies to violations:

- reviewer can edit violations tagged by curators.

✅ **Clarification (from user):** "Reviewer can change or update individual scores of each curator only... reviewer_curator_1 rating, reviewer_curator_2 rating... violations also can be edited."

⚠️ Edited Here: reviewer behavior:

- Does reviewer always edit OR sometimes only approves without edits? - Yes the reviewer can do both, either edits if required or submit/approve as it is.
- Do we store reviewer comments as well? - Yes we store reviewer comments as well so that to track the history of changes.

---

7) Auto-rater output (90% path)

For the auto-rater portion of the batch (\~90%), each row/prompt-step may have:

- `auto_rater_scores` (rubric dimensions)
- `auto_rater_confidence`
- `auto_rater_model_version`
- `auto_rater_rubric_version`

⚠️ **Edited**: is auto-rater run at prompt-level or step-level?

- If prompt-level only, we store one score per prompt.
- If step-level, we store one score per row. - Auto rater also works at step-level and provides ratings and violations where this will have only one final reviewer No seperate curator and reviewer flow.

---

8) Outputs we will build from this data

8.1 Training datasets

We will produce training datasets based on **reviewer-edited curator labels** (gold). Possible formats:

- SFT: prompt context + final answer + per-step supervision (optional)
- Preference: chosen vs rejected (if available)
- Multi-objective: rubric dimension scores as targets

8.2 Analytics datasets

We will produce analytics tables such as:

- disagreement rate (curator1 vs curator2)
- reviewer edit rate (how often reviewer changes scores)
- violation rate by pod/task/model_version
- distribution of step counts (3 vs 5 vs 10)
- quality trend over time

---

9) Implications for our pipeline design (why we model it this way)

Raw (GCS Raw → JSONL)

Raw will store **one line per row (prompt-step)**. Each JSONL record will include:

- `meta`: run_id, ingest_ts, source info, schema_hash, record_hash, row_number
- `payload`: the row payload including prompt_id, step_index, step payload, curator ratings, reviewer edits, and/or auto-rater scores.

This choice is made because the **atomic unit of evaluation is a row (prompt-step)**, not the full prompt.

---

10) Open questions to confirm (fill in)

1. Is auto-rater scoring **prompt-level** or **step-level**? - step level
2. Are `reviewer_curator_1_rating` and `reviewer_curator_2_rating` always present, or only when reviewer edits? - let them present every time.
3. Do we store the model's **final answer text** at the last step only? Or at every step snapshot? - at each row we store the full execution json in one column and the highlighed or evaluated step number/ index number will be stored in other column (suggest me if any best way to store, but my understanding is this)
4. Any PII fields to hash/redact in raw? - for the curator and reviewer we use there usernames so nothing much secret data we have here.

Notes / corrections log

- 2026-02-08: updated “row is atomic unit = prompt × step”; reviewer edits per-curator ratings.